Literature Review on Coding-Progress Analytics and Learner-Facing Dashboards

Review cut-off: 10 October 2026

Record set: 30 scholarly works; bibliographic metadata is tracked in literature_review.csv and references.bib.

1. Introduction and scope

KodBTW combines public coding-platform profile data and user-entered profile information into a dashboard. The relevant literature spans two connected areas: learner-facing learning analytics dashboards (LADs), and analytics derived from programming exercises, development traces, or online judges. These areas inform interface design, data provenance, metric interpretation, and evaluation. They do not establish that a coding-profile aggregator improves learning.

The review is deliberately bounded. It is not a review of all competitive-programming pedagogy, all commercial coding dashboards, or every learning analytics system. The 30 records include primary studies, systematic reviews, and research working-group reports relevant to dashboard design/evaluation, programming data, and online judge systems. The bibliography records where full text was discoverable and where only publisher metadata/abstract was available.

2. Review objectives and questions

The review asks: (RQ1) how learner-facing analytics dashboards select and present learner data; (RQ2) how programming education research derives evidence from code, submission, and course traces; and (RQ3) what evaluation and data-integrity gaps are relevant to a multi-platform profile application.

3. Search and source-selection method

Searches were run on 10 October 2026 using title/topic queries covering learning analytics dashboards, student-facing dashboards, programming education analytics, programming behavior data, programming online judges, and competitive programming. Sources included publisher pages (ACM, IEEE, Springer, Elsevier/Wiley, MDPI), ERIC, scholarly university repositories, and proceedings repositories such as ACM and Dagstuhl DROPS. Search results were screened for direct relevance to dashboards, programming-learning data, online judge systems, or methodology used to evaluate learner-facing analytics. Candidate records were deduplicated by title/DOI.

Included: English-language scholarly journal/conference works or research reports with a stable publisher/repository record; studies/reviews related to learner analytics, programming education, coding activity, dashboards, online judges, feedback, or student-facing metrics. Excluded: commercial product pages, non-scholarly descriptions, generic software engineering metrics unrelated to learners, and records lacking a stable bibliographic identity. A few adjacent foundations are retained where they explain the field or dashboard design.

The result is a structured narrative review of 30 records, not a PRISMA systematic review: no exhaustive database-wide query logs, independent dual screening, or full-text extraction for every paywalled record were available. Reference-level access status is explicit in the CSV. For records marked abstract/metadata-only, the review makes no claim of full-text critical evaluation and uses them only for their title, venue, abstract-level purpose, or a directly verified finding.

4. Thematic synthesis

4.1 Dashboard construction and stakeholder fit

Early dashboard work frames analytics as a translation of interaction traces into visual summaries for learners and instructors [3], [4]. Reviews group dashboard designs by audience, data source, visual form, and intended use, while documenting a persistent risk: a technically rich dashboard can remain weakly connected to learning theory or actionable learning goals [5]-[8], [10], [11]. Learner expectation studies also show that students may value planning, self-assessment, adaptive recommendations, and personalized analysis, while raising privacy and autonomy concerns [9]. These findings support giving the user clear data provenance and control; they do not justify adding recommendations or institutional monitoring without research and consent.

4.2 Learning impact and evaluation

The strongest caution comes from reviews that distinguish system availability, perceived usefulness, usage, behavior change, and learning outcomes. Early reviews found many exploratory or proof-of-concept dashboards and limited outcome evaluation [5], [6]. Later reviews report weak links between dashboard goals and evaluation measures and a need to connect design to learning processes [10], [11], [14]. The randomized Java-course study by Hellings and Haelermans is especially relevant: in a sample of 556 first-year students, dashboard access and use were associated with some intermediate online performance measures, but not final exam performance or passing; effects differed across student subgroups [12]. Therefore, system implementation alone cannot support an effectiveness claim.

4.3 Programming-process analytics

Programming education offers detailed data sources absent from ordinary LMS dashboards. Work has analyzed code snapshots and behavior patterns in open-ended tasks [17], the ways students' approaches to programming differ [18], and large-scale compiler events to identify frequent novice errors [19]. The ITiCSE working-group report synthesizes educational data mining and learning analytics in programming and emphasizes single-course/single-institution limitations and replication needs [16]. Research using multiple programming data sources argues that no single online environment captures all learner activity [20]. These studies inform the idea of a unified view, but their underlying data are generally course/IDE traces, not public profile metrics from several independent coding platforms.

4.4 Online judges and coding exercise platforms

Online judges automate submission assessment and can support course workflows [23]. More recent studies connect submission, problem order/difficulty, behavioral traces, immediate feedback, or performance models to programming instruction [21], [22], [26], [28]. These studies are closer to competitive-coding activity, but their measures are context-dependent: contest submissions, course exercises, and public profile statistics do not have identical semantics. Difficulty/order findings from contest datasets, for example, should not be treated as a universal measure of a student's topic mastery [21]. The current KodBTW implementation does not execute code, collect source code, classify DSA topics, or operate an online judge.

4.5 Automated feedback and programming instruction

Reviews of introductory programming and automated feedback describe a large body of work on novice difficulty, assessments, coding tools, and feedback mechanisms [24], [25], [30]. Automated feedback systems can support iterative exercises, but their requirements differ from a profile analytics dashboard. KodBTW's current “insights” are derived summaries over persisted platform statistics; they are not an instructional tutor, personalized recommendation model, or evaluated intervention.

5. Comparative overview

Cluster	Representative records	Data and methods	Contribution to this project	Key limitation for transfer
Dashboard reviews	[5]-[8], [10], [11], [13]-[15]	Literature synthesis of dashboard users, data, affordances, and evaluation	Informs user-facing display, evaluation discipline, privacy, and actionability	Education LMS dashboards are not the same as multi-platform coding profiles
Learner-facing dashboard studies	[9], [12]	Student expectations; randomized access/use study in a Java course	Highlights need for student-centered features and causal outcome measurement	Context and outcomes are course-specific; no direct evidence for KodBTW
Programming-process analytics	[16]-[20], [26]-[29]	Code snapshots, IDE/compiler events, online learning behavior, models, or platform logs	Supports careful distinction between coding trace and summarized profile metric	Most data sources are richer and more controlled than public profile APIs
Online judge and contest research	[21]-[23], [28]	Automated submissions, problem difficulty/order, platform activity	Provides adjacent evidence about coding practice and online judge behavior	Contest behavior is not equivalent to proficiency or long-term learning
Introductory programming / feedback	[24], [25], [30]	Literature reviews on programming teaching and automated feedback	Context for pedagogical support and the boundary of a statistics dashboard	Does not validate dashboard effectiveness or platform integration

6. Strengths and limitations of prior work

The literature offers multiple validated concepts for dashboard audiences, feedback, and data visualization; empirical programming studies provide domain-relevant behavior traces; systematic reviews reveal repeated limitations in evaluation, theoretical grounding, and context transfer. At the same time, access to full text varied in this review, some primary studies use small/course-specific populations, and the included records do not amount to a common metric standard for coding-profile aggregation. Many online-judge findings are observational, and dashboard usage can be confounded by learner motivation or prior performance. Public-profile data sources also raise terms, reliability, schema, and temporal-consistency questions that are not resolved by dashboard studies.

7. Evidence-based gap and implications for KodBTW

The bounded gap is not “no coding analytics exists.” Research documents programming analytics and learning dashboards. Rather, the reviewed evidence leaves a design and documentation need at their intersection: preserve each metric's source and time, avoid comparing semantically different metrics without normalization, distinguish unknown values from zero, and separate deployed individual analytics from proposed institutional intelligence. The implementation audit finds that KodBTW already stores source-tagged dated snapshots, uses nullable fields, filters unsupported live sources, and does not claim to synthesize absent history. This design maps to the integrity gap but is not empirically evaluated.

The project should next validate field mappings and upstream terms, exercise data quality and ownership requirements, and run a user study before making claims about usability or learning outcomes. Mentor dashboards, topic-gap detection, and intervention recommendations require their own requirements, data permissions, consent model, and evaluation.

8. Conclusion

Research supports a cautious distinction between an analytics interface and a demonstrated educational intervention. Coding-process analytics and online-judge studies provide useful measures, but their data cannot be substituted blindly for public profile statistics. KodBTW's current snapshot/provenance approach is a verifiable engineering contribution; claims of educational benefit, campus-level diagnosis, or fully supported five-platform synchronization remain unsupported by the current evidence.

9. References

The full 30-record citation list is in references.bib; the curated extraction/access register is literature_review.csv. Citation numbering follows the order in that database. Publisher or institutional links and access limitations are stored per record.